

Problem Set A — Python Foundations (Weeks 1–2)

Programming and Numerical Methods for Economics (ECNM10002)

The University of Edinburgh · School of Economics

Instructions. This problem set covers material from Weeks 1 and 2: Python basics, NumPy arrays, loops, conditionals, functions, data types, tuples, and dictionaries. Answer every question in the code cells provided. Where a question asks you to *explain*, write your answer as a comment or a Markdown cell.

```
In [ ]: import numpy as np
```

Question 1 — Lists and basic operations

A country's quarterly real GDP growth rates (annualised, %) over two years are:

Q1	Q2	Q3	Q4	Q1	Q2	Q3	Q4
1.2	0.8	-0.5	2.1	1.5	0.3	-1.1	0.9

- (a) Store these values in a Python list called `gdp_growth`.
- (b) Using only list indexing (no NumPy), extract the growth rates for the **second year** (`Q1`–`Q4`) into a new list called `year2`.
- (c) Write a `for` loop that counts how many quarters had **negative** growth. Store the count in a variable called `n_neg`.
- (d) Without using any built-in `sum()` or NumPy, write a loop that computes the **average** growth across all eight quarters.

```
In [ ]: # Your answer here
```

Question 2 — NumPy arrays and vectorised operations

Suppose two sectors of an economy produce output vectors (in billions of £) over five years:

- **Manufacturing:** `[120, 115, 118, 125, 130]`
- **Services:** `[200, 210, 205, 220, 240]`

- (a) Create two NumPy arrays, `mfg` and `svc`, from these lists.
- (b) Compute `total_output`, the element-wise sum of the two sectors for each year.
- (c) Compute `mfg_share`, the manufacturing share of total output for each year (as a proportion percentage).
- (d) Using NumPy, compute the **year-on-year growth rate** of total output for years `1960-1969`. Store `growth_total`. (Hint: the growth rate from year t to $t+1$ is $(y_{t+1} - y_t)/y_t$.)
- (e) What is the average manufacturing share? What is the maximum growth rate of total output? Use NumPy functions.

In []: `# Your answer here`

Question — Functions and economic application

- (a) Write a function `cobb_douglas(K, L, A=1, alpha=0.33)` that returns output $Y = A K^{\alpha} L^{1-\alpha}$. Use default parameter values as shown.
- (b) Evaluate the function at $K=10$, $L=10$ with default A and α . Then evaluate again with $A=2$.
- (c) Write a function `returns_to_scale(K, L, A=1, alpha=0.33, scale=2)` that:
- Computes $Y_1 = \text{cobb_douglas}(K, L, A, \alpha)$.
 - Computes $Y_2 = \text{cobb_douglas}(\text{scale} \cdot K, \text{scale} \cdot L, A, \alpha)$.
 - Returns the ratio Y_2 / Y_1 .

What does your function return for `scale=2`? Explain why, given the Cobb–Douglas exponents.

In []: `# Your answer here`

Question — Conditionals, loops, and dictionaries

An economist classifies countries by GDP per capita (in thousands of USD):

Country	GDP per capita
Argentina	10.5
United Kingdom	25.0
Nigeria	1.0
South Korea	15.0
Ethiopia	0.5

Country	GDP per capita
Germany	.

(a) Store this data in a dictionary called `gdp_pc` where keys are country names and values are per capita.

(b) Write a loop that iterates over the dictionary and classifies each country as:

- 'Low income' if GDP per capita <
- 'Middle income' if $\leq$ GDP per capita <
- 'High income' if GDP per capita $\geq$

Store the results in a new dictionary called `classification` with the same keys.

(c) Using your `classification` dictionary, count how many countries fall into each income g
Print the counts.

In []: `# Your answer here`

Question — Numerical integration and while loop

The **consumer surplus** for a linear demand curve $P(Q)=a-bQ$ above a market price P^* is the area of the triangle between the demand curve and the price line, from $Q=0$ to the equilibrium quantity $Q^*=(a-P^*)/b$.

The exact consumer surplus is: $CS = \int_0^{Q^*} [P(Q) - P^*] dQ = \frac{(a - P^*)^2}{2b}$

Set $a =$ \$, $b =$ \$, $P^* =$ \$.

(a) Compute the exact consumer surplus using the formula.

(b) Write a function `cs_riemann(a, b, p_star, n)` that approximates the consumer surplus a **left Riemann sum** with `n` rectangles. (Hint: divide $[0, Q^*]$ into `n` equal sub-intervals; sub-interval evaluate $P(Q_i) - P^*$ at the left endpoint and multiply by the width ΔQ .)

(c) Using a `while` loop, find the **smallest** value of `n` such that the Riemann approximation is within 1% of the exact answer. Print `n` and the approximation.

In []: `# Your answer here`

Question — Tuples, unpacking, and a mini proje

A bond pays fixed annual coupons. Write a function `bond_value(face, coupon_rate, r, T)` that:

- Computes the **present value** of all coupon payments: $PV_{\text{coupons}} = \sum_{t=1}^T \frac{C}{(1+r)^t}$, where $C = \text{face} \times \text{coupon_rate}$.
- Computes the **present value** of the face value repaid at maturity: $PV_{\text{face}} = \frac{\text{face}}{(1+r)^T}$.
- Returns a **tuple** `(pv_coupons, pv_face, total_pv)` where `total_pv = pv_coupons + pv_face`.

(a) Write the function.

(b) Call it with `face=1000`, `coupon_rate=0.05`, `r=0.03`, `T=10`. Unpack the result into variables and print each.

(c) What happens to the total bond value if the market interest rate `r` rises to `0.05`? Is the bond trading at a premium or a discount? Explain in one sentence.

In []: `# Your answer here`

End of Problem Set A.